ASSIGNMENT ON STRUCTURED PROGRAMMING LAB

Emergency Potable Water Allocation for a Flood-Affected Municipality

Student Name: 1. AL ARAF BIN ALAM
2. MARUF ABID JOTI

Student ID: 1. 26103163
2. 26103239

Programme: Bachelor of Computer Science and Engineering (BCSE)

Course Code: CSC 1384

Course Title: Structured Programming LAB

Section: D

Course Instructor: Abdullah Mohammad Sakib

Submission Date: 6 September 2026

2. Problem Analysis

The assignment addresses an emergency potable-water allocation problem created by severe flooding. A temporary municipal distribution centre has 13,500 litres of potable water available for six affected service zones, while the total requested quantity is greater than the available supply. Therefore, the programme must decide how limited water should be distributed without relying on a fixed order of request. The official task specifically requires the student to identify relevant decision factors and formulate an allocation strategy that balances emergency need, fairness and efficient utilisation of water.

The six service zones differ in service type, requested quantity, minimum requirement, distribution loss and previous waiting cycles. These attributes create competing priorities. Hospitals and emergency services require dependable minimum supply; shelters serve displaced people; residential areas may have accumulated waiting time; and higher distribution losses reduce the amount that finally reaches the destination. A useful computational solution must therefore consider more than one variable at the same time.

2.1 Original Data Set

Zone ID	Service Zone	Type	Requested (L)	Minimum (L)	Loss	Waiting
Z01	Municipal Hospital	Hospital	4000	3000	2%	0
Z02	Central Flood Shelter	Emergency Shelter	3500	2500	4%	1
Z03	Ward 3	Residential	4500	2000	8%	2
Z04	Ward 5	Residential	3800	1800	12%	3
Z05	School Emergency Shelter	Emergency Shelter	2600	1600	5%	1
Z06	Fire and Emergency Service	Emergency Service	2000	1500	1%	0

The total requested amount is 20,400 L. Because only 13,500 L is available in the original case, the programme cannot satisfy all full requests simultaneously. The problem is therefore one of constrained allocation: the programme must decide which zones should be served first and how much can be released while respecting the available quantity.

2.2 Major Decision Factors

- Requested quantity indicates the total amount a zone would ideally receive. Larger requests represent larger unresolved demand when supply is constrained.
- Minimum requirement represents a lower service threshold. The allocation logic attempts to satisfy this threshold before giving water to lower-priority zones, subject to availability.
- Service-zone type captures urgency. The implementation assigns higher base urgency to hospitals and emergency services, a slightly lower value to emergency shelters, and a lower value to residential areas.
- Previous waiting cycles provide a fairness component. A zone that has remained underserved for more cycles receives additional priority.
- Distribution loss represents water that is lost between release and destination. A high loss percentage makes a release less efficient and therefore creates a small penalty in the priority score.
- Unresolved demand is represented in the priority formula through the gap between requested water and minimum requirement. A larger gap increases the demand component.
- Shortage is the remaining difference between the requested quantity and the effective water actually delivered after losses. It is used after allocation to classify final service condition and identify the highest unresolved requirement.

2.3 Conflicting Requirements

A first-come-first-served approach would ignore differences in urgency and accumulated waiting. Equal distribution would treat a hospital, an emergency shelter and a residential area as if they had the same emergency role. A fixed priority list would be rigid and would not respond appropriately when the input values

change. The assignment instead asks for a strategy that remains functional when values are changed, so the decision logic must be calculated from the supplied data rather than hard-coded for one dataset.

- Emergency requirement versus fairness: critical services may need immediate water, but repeatedly starving residential zones is unfair.
- Complete supply versus wider coverage: fully satisfying a small number of zones can leave other affected populations unserved, while spreading water too thinly can leave critical facilities below minimum.
- Distribution efficiency versus social requirement: a high-loss zone is less efficient to supply, but excluding it solely because of loss would ignore the people depending on that zone.
- Previous waiting versus new emergency: a long waiting period can justify increased priority, but a newly urgent service may still need a high base priority.
- Accuracy versus computational efficiency: a decision model should represent the relevant factors without introducing unnecessary repeated calculations or redundant data handling.

2.4 Problem Constraints

- Total water allocated must never exceed the available water.
- All six zones must be processed systematically.
- The method must continue working when input values are changed.
- Ties in priority must be resolved consistently.
- Distribution loss must be reflected in the amount released and in the effective delivered amount.
- The final report must explain the strategy, implementation and validation evidence.

3. Proposed Solution

The programme uses a weighted priority score followed by a descending sort. This strategy is a design choice made because the assignment intentionally does not prescribe a fixed priority formula. The actual C source defines the score using urgency, waiting, the requested-minus-minimum demand gap, and a small loss penalty.

3.1 Priority Formula

Priority = Urgency + Waiting Score + Demand Score - Loss Penalty

Waiting Score = Waiting Cycles x 2.0

Demand Score = (Requested Water - Minimum Requirement) / 500.0

Loss Penalty = Distribution Loss (%) x 0.10

Therefore:
Priority = Urgency + (Waiting Cycles x 2.0)
+ [(Requested - Minimum) / 500]
- (Loss x 0.10)

The constants 2.0, 500.0 and 0.10 are not provided by the assignment specification; they are the weighting decisions encoded in the submitted programme. The design gives service type the strongest direct influence, adds a meaningful but controlled reward for waiting, increases priority for zones with a larger gap above their minimum requirement, and slightly reduces priority when distribution loss is high.

3.2 Urgency Values

Service Type	Urgency Score	Rationale in the Programme
Hospital	10	Highest emergency importance
Emergency Service	10	Highest emergency importance
Emergency Shelter	8	High emergency/social need
Residential	5	Lower base urgency but affected by waiting/demand

3.3 Worked Priority Calculations

Z04 - Ward 5

Urgency = 5; Waiting = 3; Loss = 12%; Requested = 3800; Minimum = 1800.
Priority = 5 + (3 x 2.0) + ((3800 - 1800)/500) - (12 x 0.10)
= 5 + 6 + 4 - 1.2
= 13.80.

Z03 - Ward 3

Urgency = 5; Waiting = 2; Loss = 8%; Requested = 4500; Minimum = 2000.
Priority = 5 + (2 x 2.0) + ((4500 - 2000)/500) - (8 x 0.10)
= 5 + 4 + 5 - 0.8
= 13.20.

Z01 - Municipal Hospital

Urgency = 10; Waiting = 0; Loss = 2%; Requested = 4000; Minimum = 3000.
Priority = 10 + 0 + ((4000 - 3000)/500) - (2 x 0.10)
= 10 + 2 - 0.2
= 11.80.

3.4 Priority Order for the Original Case

Order	Zone	Priority
1	Z04 - Ward 5	13.80
2	Z03 - Ward 3	13.20
3	Z01 - Municipal Hospital	11.80
4	Z02 - Central Flood Shelter	11.60
5	Z05 - School Emergency Shelter	11.50
6	Z06 - Fire and Emergency Service	10.90

3.5 Distribution-Loss Formulas

Full release requirement = Requested Water / (1 - Loss% / 100)

Minimum release requirement = Minimum Water / (1 - Loss% / 100)

Division by the retained fraction is necessary because the amount released at the distribution centre is larger than the quantity finally received. For example, with 12% loss, only 88% of the released amount reaches the destination. Therefore, to deliver exactly 3,800 L, the release must be 3,800 / 0.88 = 4,318.18 L rather than 3,800 x 0.88.

Loss Amount = Allocated Water x Loss% / 100

Effective Water Delivered = Allocated Water - Loss Amount

Remaining Shortage = Requested Water - Effective Water Delivered

The programme then prevents a negative shortage by setting it to zero when the computed value is below zero. Service status is FULL when effective delivery reaches the request, MINIMUM when it reaches the minimum requirement, BELOW MIN when some water is delivered but the minimum is not reached, and UNSERVED when no water is delivered.

3.6 Worked Distribution Example

For Z04, the original requested quantity is 3,800 L and the distribution loss is 12%. The release needed for full delivery is 3,800 / (1 - 0.12) = 3,800 / 0.88 = 4,318.18 L. The resulting loss is 4,318.18 x 0.12 = 518.18 L,

leaving approximately 3,800.00 L delivered. The same principle is applied to the minimum requirement: 1,800 / 0.88 = 2,045.45 L.

3.7 Allocation Procedure

1. Reset previously calculated fields.
2. Calculate priority for every zone.
3. Sort zones in descending priority order.
4. For each zone, calculate the release required to satisfy the full requested quantity.
5. If enough water remains, allocate the full release requirement and reduce the available quantity.
6. Otherwise, calculate the release required to satisfy the minimum requirement; if that is possible, allocate the minimum release requirement.
7. If even the minimum release requirement is larger than the remaining water, allocate whatever water remains and then exhaust the supply.
8. After allocation, calculate loss, effective delivery, shortage and service condition for every zone.

3.8 Tie-Breaking Rule

When two priorities are exactly equal, the sorting logic first compares waiting cycles and places the zone with more waiting cycles ahead. If waiting cycles are also equal, the programme compares minimum requirements and places the zone with the larger minimum requirement ahead. This creates a deterministic ordering without requiring a separate fixed priority list.

4. Programme Design

4.1 Data Organisation

The programme defines a single structure, struct Zone, to keep related information together. Each element of the Zone array represents one service zone and stores both original inputs and calculated results. This representation avoids maintaining multiple independent arrays that could become mismatched.

Field	Type	Purpose
id	char[10]	Zone identifier such as Z01
name	char[40]	Service-zone name
type	char[25]	Service category used for urgency score
requested	float	Requested water in litres
minimum	float	Minimum required water in litres
loss	float	Distribution loss percentage
waiting	int	Previous waiting cycles
priority	float	Calculated priority score
order	int	Final allocation order
allocated	float	Water released from the centre
lossAmount	float	Water lost during distribution
delivered	float	Effective water received
shortage	float	Unresolved part of the request
status	char[20]	Final service condition

4.2 Function Decomposition

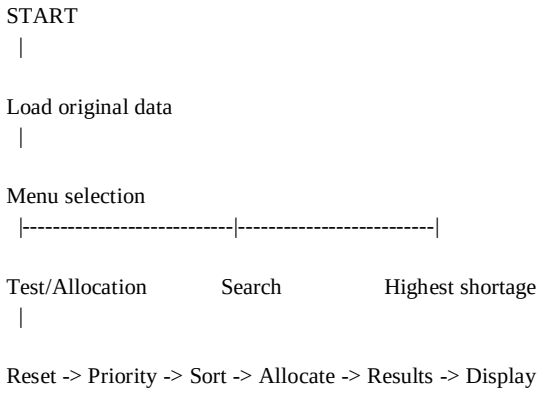
Function	Purpose	Inputs/Outputs
loadOriginalData()	Loads the six original zones and resets calculated fields.	Zone array; no return value
resetResults()	Initialises calculated fields before each run.	Zone array
getUrgency()	Maps service type to urgency score.	Type string -> float score
calculatePriority()	Computes priority for all zones.	Zone array
sortZones()	Sorts zones by priority and resolves ties.	Zone array
allocateWater()	Allocates available water according to priority and release requirements.	Zone array, available water

calculateResults()	Computes loss, delivery, shortage and status.	Zone array
runAllocation()	Coordinates one complete allocation run.	Zone array, available water
displayResult()	Prints the table, order and summary.	Zone array, available water
searchZone()	Finds a zone by ID and displays its information.	Zone array; user input
findHighestShortage()	Identifies the largest unresolved requirement.	Zone array
case1()...case7()	Implements the required validation scenarios.	Original zone data
main()	Displays menu, accepts choices and calls the selected operation.	User input

4.3 Pseudocode

```
BEGIN
  Load original zone data
  REPEAT
    Display menu
    Read choice
    IF choice selects an allocation/test case THEN
      Copy original zone data to working array
      Reset calculated fields
      FOR each zone
        calculate urgency, waiting score, demand score and loss penalty
        calculate priority
      END FOR
      Sort zones by descending priority
      Resolve equal priority using waiting cycles, then minimum requirement
      For each zone in priority order
        calculate full release requirement
        calculate minimum release requirement
        allocate full requirement when possible
        otherwise allocate minimum requirement when possible
        otherwise allocate remaining water
        stop when remaining water is zero
      END FOR
      Calculate loss, delivered quantity, shortage and status
      Display results
    ELSE IF choice is search THEN
      run original allocation and search by zone ID
    ELSE IF choice is highest shortage THEN
      run original allocation and find largest shortage
    ELSE IF choice is exit THEN
      terminate loop
    ELSE
      display invalid choice
  UNTIL choice = Exit
END
```

4.4 Programme Flow



```
|

Return to menu
|

Exit when choice = 10
```

4.5 Core Allocation Logic from the Source

```
fullRequirement = zone[i].requested / (1.0 - zone[i].loss / 100.0);
minimumRequirement = zone[i].minimum / (1.0 - zone[i].loss / 100.0);

if (available >= fullRequirement) {
   zone[i].allocated = fullRequirement;
   available = available - fullRequirement;
}
else if (available >= minimumRequirement) {
   zone[i].allocated = minimumRequirement;
   available = available - minimumRequirement;
}
else {
   zone[i].allocated = available;
   available = 0;
}
```

This block is the main constraint-enforcement point. Because the working variable available is reduced after every successful allocation and becomes zero when the remainder is exhausted, the total released quantity cannot exceed the starting available amount within a run.

5. Optimisation Considerations

The programme is designed as structured procedural code rather than as a single monolithic function. Optimisation is therefore considered in terms of avoiding unnecessary storage, avoiding repeated re-entry of the same original data, reusing calculated information, and keeping the number of processing steps appropriate to six zones.

- The original dataset is loaded once by loadOriginalData(). Test cases copy the original structures into a working array, so the source values do not need to be typed again for every test.
- The struct Zone representation stores related values together and prevents duplication across separate arrays such as ids[], requests[], minimums[] and losses[].
- Calculated information such as priority, allocation, loss, delivery and shortage is stored in the same structure and reused by displayResult(), searchZone() and findHighestShortage().
- Function decomposition avoids repeating the same allocation algorithm across all test cases. runAllocation() centralises the common processing sequence.
- The programme uses nested-loop sorting. With six zones, this is computationally small, and the implementation is straightforward within the scope of a structured programming laboratory.
- Search uses a linear scan through the six zones. This is O(n) time and is appropriate for the small fixed dataset.
- The bubble-sort-like ordering implementation is O(n^2) in the worst case. For the current six-zone input this is acceptable and keeps the algorithm transparent for learning purposes.
- Memory use is modest: the programme keeps one original array and temporary working arrays for each case, rather than storing duplicated tables of unrelated scalar values.
- The design favours correctness, readability and reuse rather than trying to minimise the number of source-code lines.

One practical limitation remains: TOTAL_ZONES is defined as 6, so the current implementation is optimised around the assignment dataset rather than being a fully dynamic system. That is a scope decision consistent with the supplied assignment, but it limits scalability without source-code modification.

6. Implementation and Results

6.1 Headers, Constants and Structure

```
#include <stdio.h>
#include <string.h>
#define TOTAL_ZONES 6

struct Zone {
    char id[10];
    char name[40];
    char type[25];
    float requested;
    float minimum;
    float loss;
    int waiting;
    float priority;
    int order;
    float allocated;
    float lossAmount;
    float delivered;
    float shortage;
    char status[20];
};
```

stdio.h supplies console input/output such as printf() and scanf(). string.h supplies string operations used by the implementation, including strcpy() for loading names and strcmp() for category and ID comparisons. TOTAL_ZONES keeps the array and processing loops consistent with the six-zone assignment dataset.

6.2 Priority Calculation Implementation

```
urgency = getUrgency(zone[i].type);
waitingScore = zone[i].waiting * 2.0;
demandScore = (zone[i].requested - zone[i].minimum) / 500.0;
lossPenalty = zone[i].loss * 0.10;
zone[i].priority = urgency + waitingScore + demandScore - lossPenalty;
```

The source calculates each component directly from the current zone values. Because the calculation is performed at runtime, changing waiting cycles, loss, requested water, minimum requirement or service type changes the resulting priority without rewriting the allocation order.

6.3 Sorting, Selection and Conditional Logic

```
if (zone[j].priority < zone[j + 1].priority) {
    // swap for descending priority
}
else if (zone[j].priority == zone[j + 1].priority) {
    // compare waiting cycles, then minimum requirement
}
```

Nested for-loops implement the ordering process. The if/else chain handles the three allocation states: enough for the complete request, enough for the minimum requirement, or only enough to release the remaining water. The switch statement in main() dispatches the selected case or utility function.

6.4 Result Classification

```
if (zone[i].delivered >= zone[i].requested)
   status = "FULL";
else if (zone[i].delivered >= zone[i].minimum)
   status = "MINIMUM";
else if (zone[i].delivered > 0)
   status = "BELOW MIN";
else
   status = "UNSERVED";
```

The display function also calculates aggregate totals: total requested, total allocated, total effective delivery, total unresolved requirement, remaining water, and the number of zones in each service-condition category.

6.5 Original Case Result

The supplied output evidence for Case 1 uses the original six-zone dataset and 13,500 L. The programme orders the zones as Z04, Z03, Z01, Z02, Z05 and Z06. It releases the full requirement for the first three zones and, when the remaining quantity becomes too small even for the next minimum release requirement, assigns the remaining water to the next zone. The evidence reports 13,500.00 L allocated, 12,500.53 L effectively delivered, 7,899.47 L unresolved, and no remaining water.

Case 1 summary	Value
Available water	13,500 L
Total requested	20,400 L
Total allocated	13,500 L
Total effective delivered	12,500.53 L
Total unresolved requirement	7,899.47 L
Remaining water	-0.00 L (display rounding)
Full requirement satisfied	2 zones
Minimum requirement satisfied	1 zone
Below minimum	1 zone
Completely unserved	2 zones

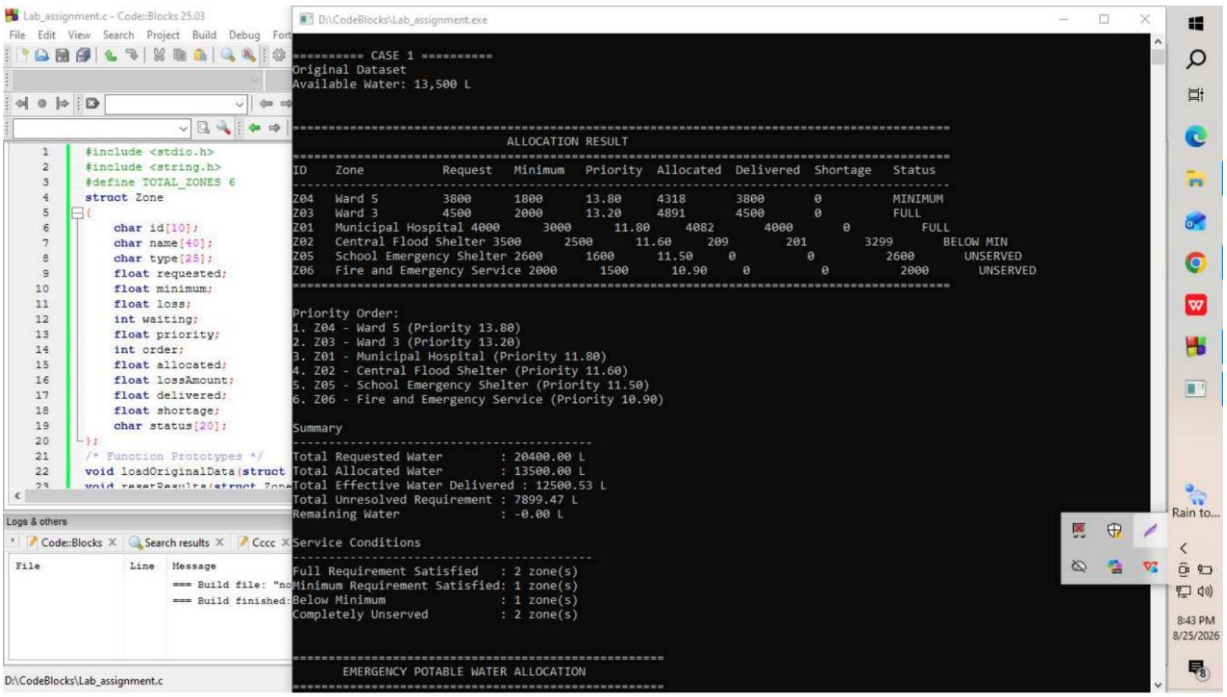


Figure 6.1: Supplied output evidence for Case 1 - original dataset with 13,500 L.

7. Testing and Validation

The assignment requires validation under six specified conditions plus at least one independent test. For each test, the report records the test condition, input or modifications, expected behaviour, actual result from the supplied evidence, and the pass/fail assessment. The output screenshots in this section are reproduced from the supplied evidence PDF rather than recreated.

Case 1: Original Dataset + 13,500 L

Item	Details
Test condition	Run the original dataset without modifying zone attributes.
Input / modified values	Available = 13,500 L
Expected result	Total allocation must not exceed 13,500 L; priority order should be calculated from the original data; some zones should remain unresolved because total request exceeds supply.
Actual result	Evidence reports 13,500.00 L allocated, 12,500.53 L delivered, 7,899.47 L unresolved, and zero remaining water. Priority order: Z04, Z03, Z01, Z02, Z05, Z06.
Assessment	PASS

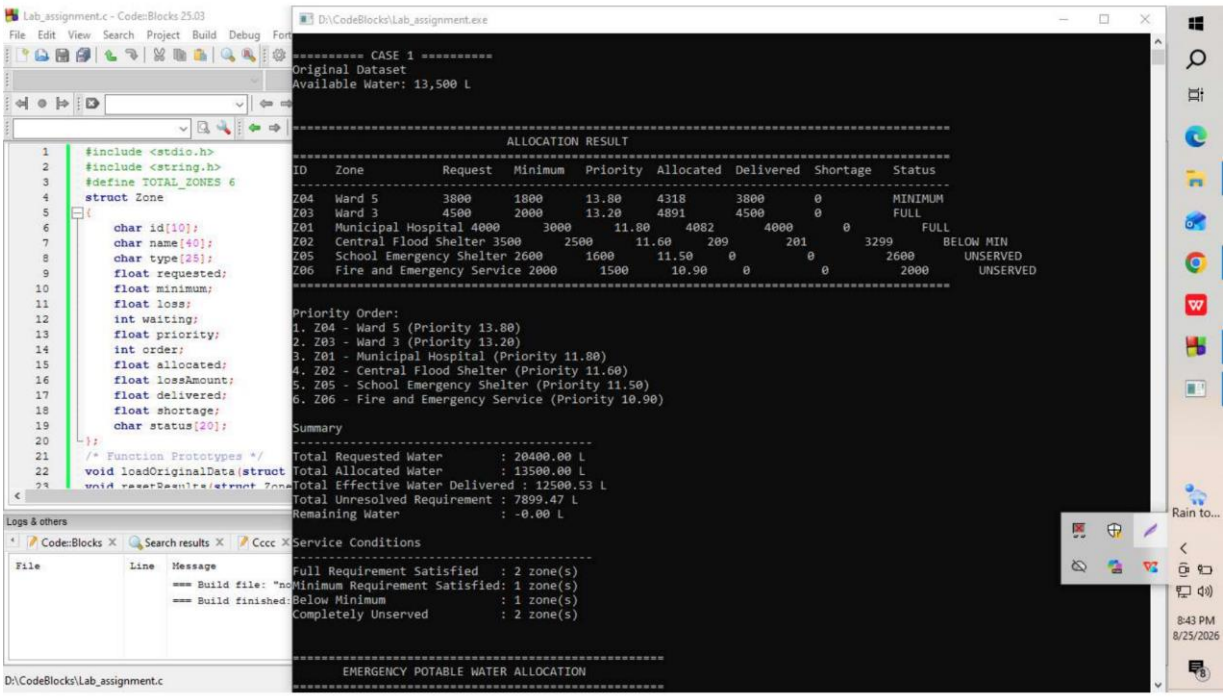


Figure 7.1: Supplied programme output evidence for Case 1.

Case 2: Enough Water for All Complete Requests

Item	Details
Test condition	Use sufficient water for all six requested quantities after accounting for distribution loss.
Input / modified values	Available = 22,194.00 L (programme-generated; includes 500 L extra).
Expected result	Every zone should receive its complete requested quantity; unresolved requirement should become zero; some water should remain.
Actual result	Evidence reports 21,694.00 L allocated, 20,400.00 L delivered, zero unresolved requirement and 500.00 L remaining. Five zones are marked FULL and Z04 is displayed as MINIMUM even though delivered equals the displayed request.
Assessment	PASS WITH NOTE

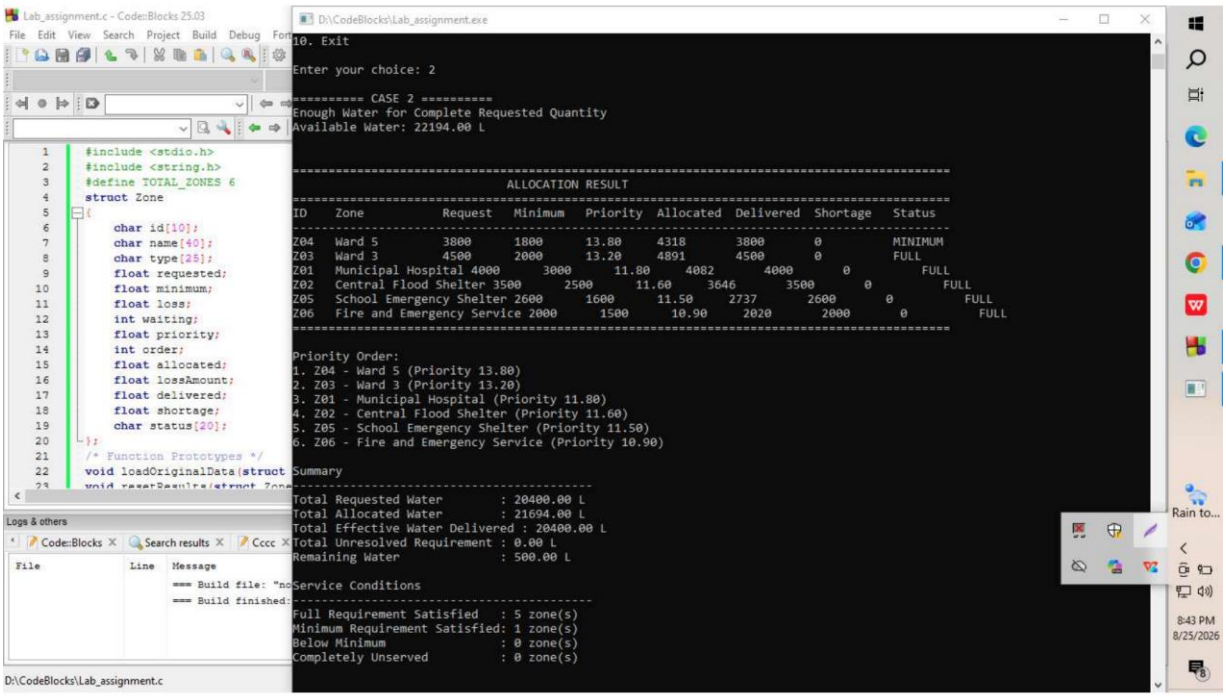


Figure 7.2: Supplied programme output evidence for Case 2.

Implementation note: The evidence displays Z04 as MINIMUM while its effective delivered quantity is shown as 3,800 L against a 3,800 L request. This likely reflects floating-point boundary behaviour: the internal value can be fractionally below the displayed whole-number result even though printf() rounds it to 3,800.00/3,800. The output therefore passes the scenario-level test but reveals a precision sensitivity at the status boundary.

Case 3: Below Combined Minimum Requirement

Item	Details
Test condition	Provide less water than the sum of the minimum requirements.
Input / modified values	Available = 8,000 L; combined minima = 12,400 L.
Expected result	Not all minimum requirements can be satisfied. Supply should be exhausted, with at least some zones below minimum or unserved.
Actual result	Evidence reports 8,000.00 L allocated, 7,277.75 L delivered, 13,122.25 L unresolved, and 0.00 L remaining. Two zones satisfy minimum, one is below minimum and three are unserved.
Assessment	PASS

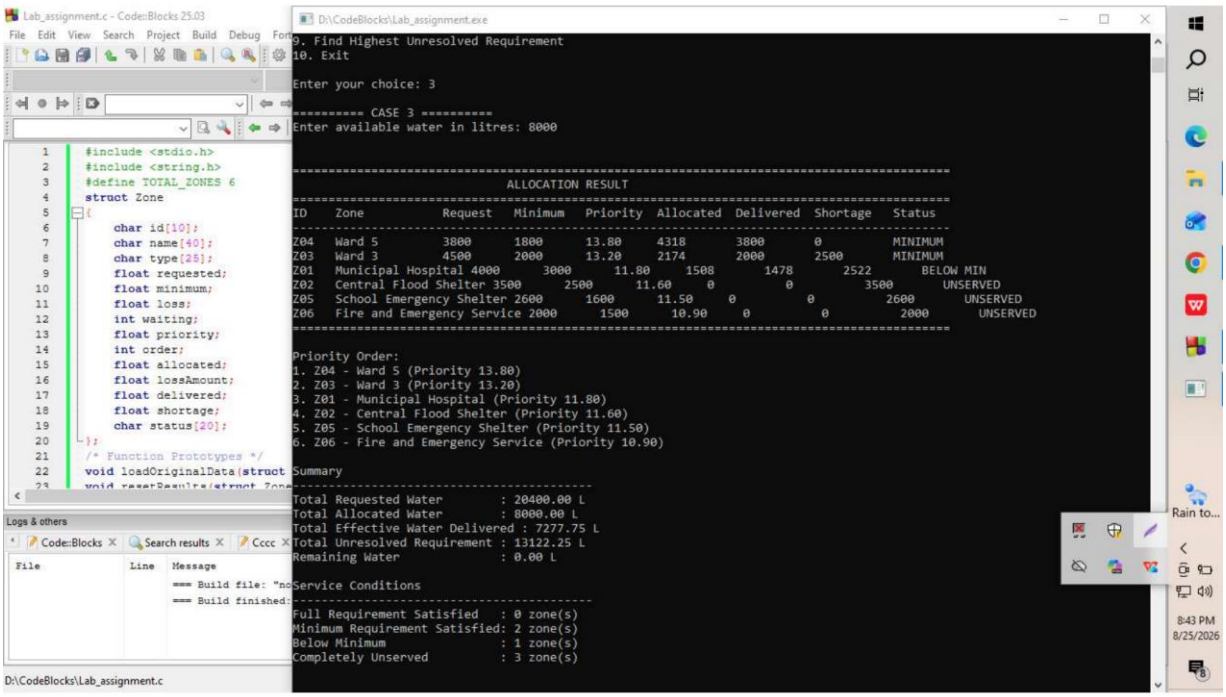


Figure 7.3: Supplied programme output evidence for Case 3.

Case 4: Equal Priority

Item	Details
Test condition	Modify Z03 and Z04 so their priority scores are equal.
Input / modified values	Z03 and Z04 modified to requested 4,000 L, minimum 2,000 L, loss 10%, waiting 3 cycles.
Expected result	The programme should calculate equal priority and resolve the tie according to its tie-breaking rule.
Actual result	Evidence shows Z03 and Z04 both have priority 14.00 and they occupy orders 1 and 2. Their other compared values are also equal, so the displayed order demonstrates deterministic processing without changing the priority score.
Assessment	PASS

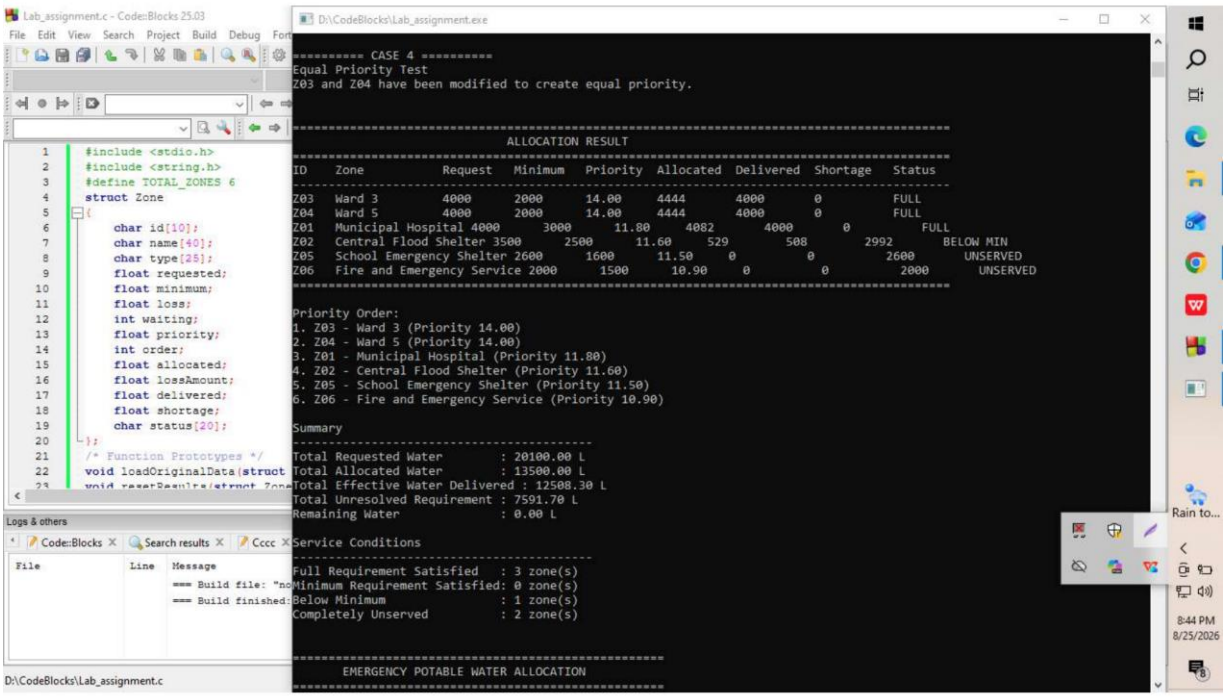


Figure 7.4: Supplied programme output evidence for Case 4.

Case 5: Changed Distribution Loss for a Residential Zone

Item	Details
Test condition	Use the Case 5 interaction to modify a residential zone loss.
Input / modified values	Selected Z03; original loss 8.00%; entered new loss 4.00%.
Expected result	A change in loss should alter the loss penalty, release requirement and effective-delivery results. The required specification describes an increase, but the supplied evidence actually decreases the loss.
Actual result	Evidence shows Z03 priority changes from 13.20 to 13.60, allocated water changes to 4,688 L, delivered water is 4,500 L and status is FULL. The change clearly affects the result, but the input is 8% -> 4%, not an increase.
Assessment	NOT FULLY COMPLIANT

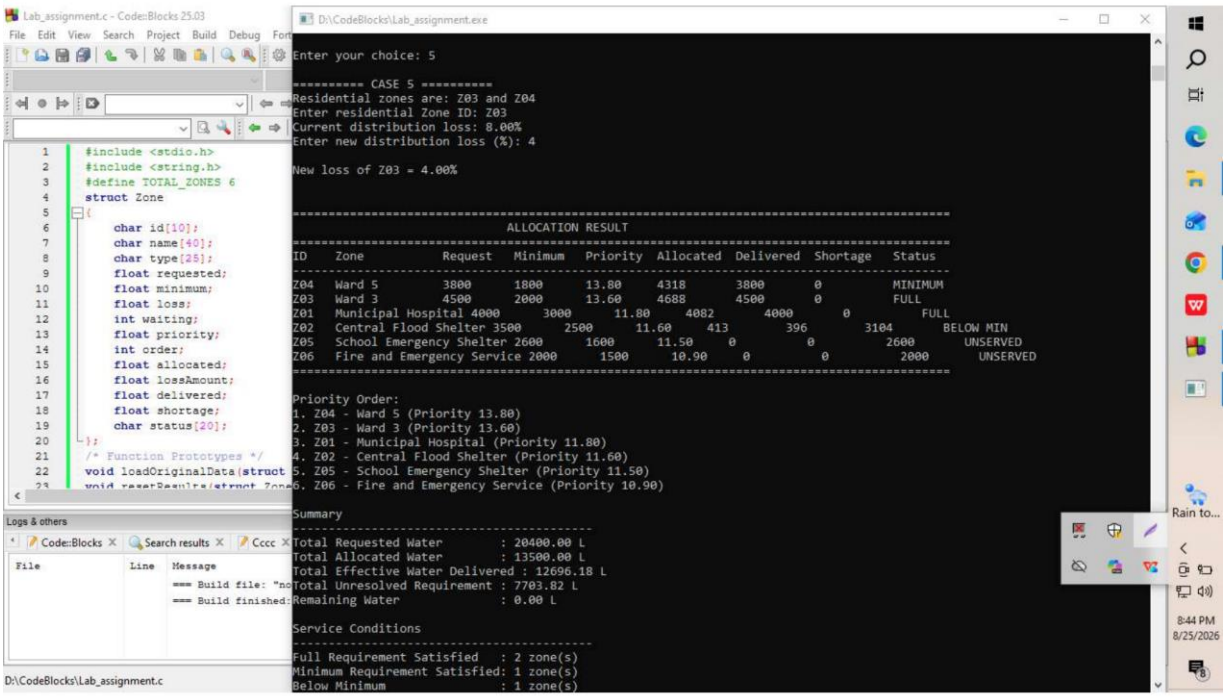


Figure 7.5: Supplied programme output evidence for Case 5.

Validation note: The official assignment asks for an increase in the distribution loss of one residential zone. The supplied evidence instead shows Z03 changing from 8.00% to 4.00%, which is a decrease. The programme reacts correctly to the entered value, but this particular evidence does not demonstrate the exact direction requested by the assignment. It should therefore be treated as a partial validation of the loss-sensitivity mechanism rather than complete proof of the specified increased-loss scenario.

Case 6: Increased Waiting Period of an Underserved Zone

Item	Details
Test condition	Select an underserved zone from Case 1 and increase its waiting cycles.
Input / modified values	Selected Z02; waiting cycles changed from 1 to 4.
Expected result	The waiting component should increase the zone priority and may change its allocation order/result.
Actual result	Evidence shows Z02 priority increases to 17.60 and it moves to order 1. It receives 3,646 L, delivers 3,500 L and becomes FULL.
Assessment	PASS

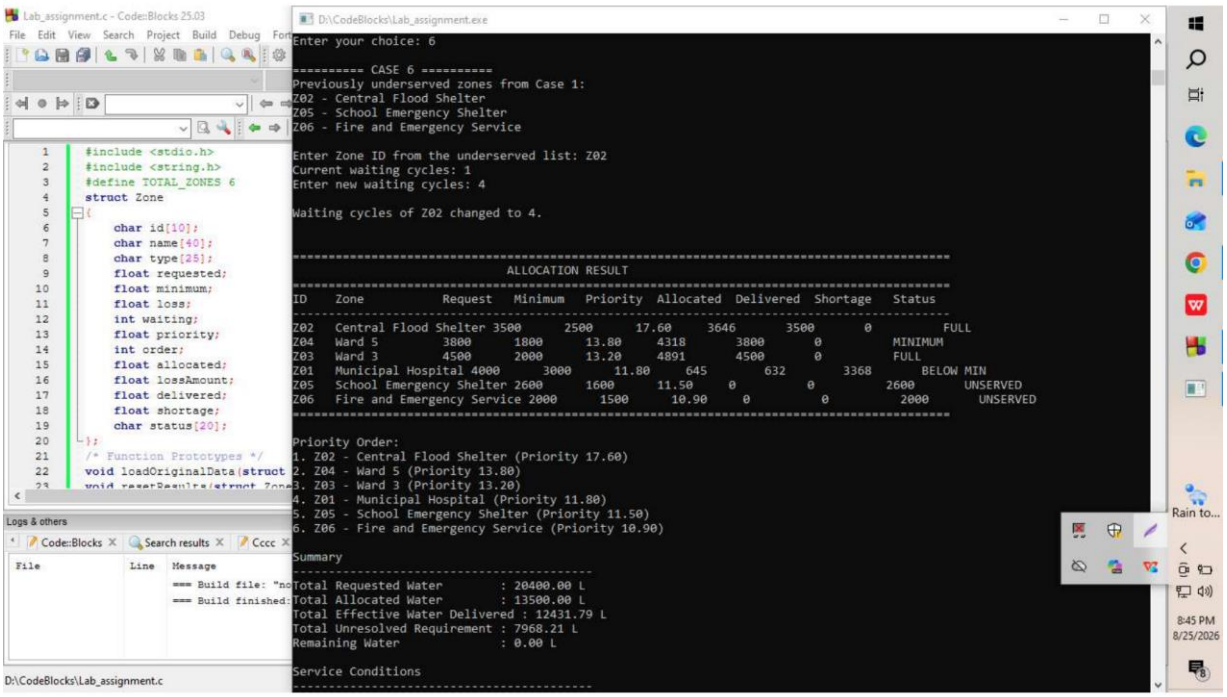


Figure 7.6: Supplied programme output evidence for Case 6.

Case 7: Independent Test - 5,000 L Available

Item	Details
Test condition	Run an independent test with a manually chosen available quantity.
Input / modified values	Available = 5,000 L.
Expected result	The programme should respect the 5,000 L cap and produce consistent priority-based allocation results.
Actual result	Evidence reports 5,000.00 L allocated, 4,427.27 L delivered, 15,972.73 L unresolved and 0.00 L remaining. Z04 is the only zone shown as satisfying minimum; four zones are unserved.
Assessment	PASS

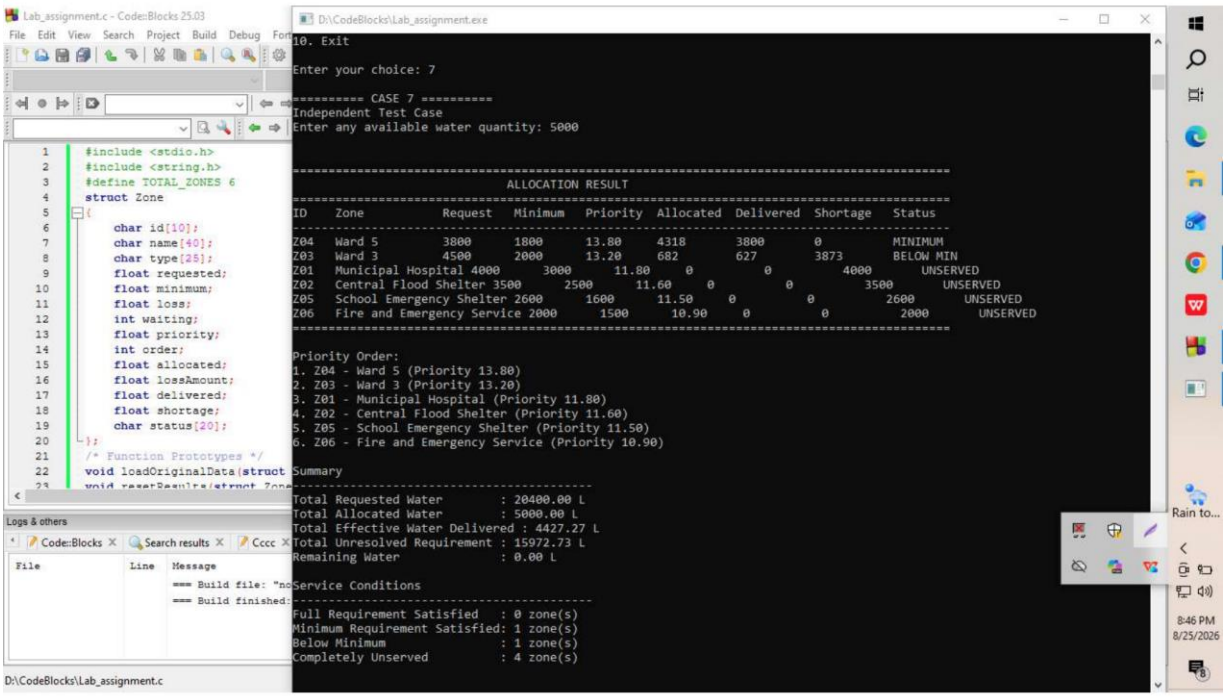


Figure 7.7: Supplied programme output evidence for Case 7.

7.1 Testing Summary

Case	Condition met?	Supply constraint respected?	Key observation	Overall assessment
1	Yes	Yes	Original constrained allocation works.	PASS
2	Yes	Yes	All requests resolved; 500 L remains. Boundary status rounding note.	PASS WITH NOTE
3	Yes	Yes	Below combined minimum handled; supply exhausted.	PASS
4	Yes	Yes	Equal priorities produced for Z03/Z04.	PASS
5	No - direction differs	Yes	Loss changed and result responded, but loss decreased rather than increased.	NOT FULLY COMPLIANT
6	Yes	Yes	Waiting increase moved Z02 to first priority.	PASS
7	Yes	Yes	Independent 5,000 L stress condition behaved consistently.	PASS

7.2 Overall Validation Discussion

Across the supplied evidence, the programme demonstrates that it can re-evaluate priority and allocation when the available supply or selected zone attributes change. Cases 2 and 3 show opposite supply conditions; Case 4 tests equal priorities; Case 5 demonstrates that changing loss changes priority and allocation behaviour, although the evidence uses a lower rather than higher loss; Case 6 demonstrates sensitivity to waiting cycles; and Case 7 provides an independently selected low-supply condition. The screenshots provide direct evidence of the executed programme outputs for these scenarios.

8. Limitations

- The weighting constants in the priority formula are manually selected. Different constants could produce a different ordering, so the score should be treated as a justified heuristic rather than a mathematically unique optimum.
- TOTAL_ZONES is fixed at 6. Supporting a different number of zones requires changes to the constant and associated data-loading design.
- The distribution-loss model assumes a fixed percentage for each zone during a run. It does not model time-varying losses, transport distance or separate loss events.
- The programme has no real-time sensor or logistics data. It operates only on the supplied numerical inputs and user-entered changes for the test cases.
- No geographical routing, travel time, road accessibility or delivery capacity constraints are considered, even though these could matter in a real flood response.
- The model assumes requirements and zone properties remain constant throughout one allocation cycle.
- Tie-breaking is simplified to waiting cycles followed by minimum requirement; other fairness indicators could be used.
- Floating-point arithmetic can create boundary effects when a displayed result appears equal to a requirement but the internal value is fractionally smaller, as suggested by the Case 2 status for Z04.
- The current programme is intended for structured-programming learning and therefore does not include advanced optimisation methods, external databases, network communication or persistent storage.

9. Conclusion

The submitted C programme provides a structured solution to the emergency potable-water allocation problem by converting several competing considerations into a single runtime priority score. The approach gives high urgency to hospitals and emergency services, rewards zones that have waited longer, increases priority when the requested-to-minimum gap is larger, and applies a small penalty for distribution loss. After priorities are calculated, the programme orders zones and allocates water while explicitly accounting for loss between the distribution centre and destination.

The implementation demonstrates core structured programming concepts, including structures, arrays, functions, iteration, selection, string operations, searching, sorting, user input and formatted output. The separation of priority calculation, sorting, allocation, result calculation and presentation makes the programme easier to understand and reuse across the test cases. The validation evidence shows that the programme responds to different supply levels, equal priorities, altered loss and waiting values, and an independent low-supply test. The main qualification is that the supplied Case 5 evidence decreases the loss percentage rather than increasing it as requested, so that particular test should be rerun with a genuine increase for complete assignment compliance.

Overall, within the scope of the assignment, the programme is a reasonable and explainable decision-support solution: it respects the available-water constraint, incorporates distribution loss into release calculations, and maintains a transparent priority-based allocation process that changes when input conditions change.

10. References

[1] E. Balagurusamy, Programming in ANSI C, McGraw-Hill Education.

[2] OpenAI, ChatGPT.